

This homework focuses on building a simple **genome assembler**, relying on a node-centric de Bruijn graphs of order k .

The file `readfa.py` contains the `readfq` function to iterate over the genomic sequences of a FASTA file.

```
from xopen import xopen
from readfa import readfq

with xopen(file_path_gz) as fasta:
    for _, seq, _ in readfq(fasta):
        # process seq
```

(1) ■ The de Bruijn graph

We start by building a perfect dBG.

► **Question 1.** *What is the minimal information you need to represent a de Bruijn Graph (dBG)? Propose a data structure (in Python) to store such a graph.*

► **Question 2.** *Implement a function `create_dbg` that builds a dBG of order k , given a sequence `seq` and the size k of k -mers. Remember that you need to properly handle k -mers belonging to the forward and reverse strand of a genome.*

► **Question 3.** *Use the `networkx` package to plot a few toy de Bruijn graphs, highlighting the effect of repetitions within genomes, or the impact of one letter change on the graph topology.*

(2) ■ Curation of the de Bruijn graph

In this section you are going to build a dBG of different sequencing experiments. More precisely, a $20\times$ coverage of 100-bp reads has been simulated with different characteristics an *E. coli* genome fragment.

The files are available as (gzipped) FASTA files within the homework directory.

- (i) `ecoli_sample_perfect_reads_forward.fasta.gz`: perfect reads, forward strand only
- (ii) `ecoli_sample_perfect_reads.fasta.gz`: perfect reads, both forward/reverse strands
- (iii) `ecoli_sample_reads_001.fasta.gz`: 0.1% error, both strands
- (iv) `ecoli_sample_reads_01.fasta.gz`: 1% error, both strands

► **Question 4.** *Ensure `create_dbg` takes now as input a (possibly compressed) FASTA file, and uses all the sequences it contains to build the de Bruijn graph.*

For each of the samples, build the graph for $k = 31$.

► **Question 5.** *How many kmers are processed during construction? What is the distribution of their multiplicity? How many of them remain in the graph?*

► **Question 6.** *Refine your assembler by adding an abundance threshold t , that is, the minimum number of times a k -mer has to occur in the input sequences to be stored within the graph.*

► **Question 7.** *Try different values for t on the erroneous reads, what do you observe? Can you propose a good value for t , justifying your choice?*

(3) ■ Unitigs of the de Bruijn graph

Remember that a *unitig* is defined as the genomic sequence that can be spelled by a maximal non-branching path in a sequence graph.

► **Question 8.** Create a method/function `unitig_from`, that takes as input a *DBG* of order k and a k -mer s , and returns the unitig containing s .

► **Question 9.** Apply this method on the previously created de Bruijn graphs using as input the 31-mer

`CGCTCTGTGTGACAAGCCGGAAACCGCCCAG.`

What is the size of the recovered unitig, for each read dataset and each $t \in \{1, 2, 3, 4\}$? Did you observe something else (if so, how would you explain it)?

► **Question 10.** Create a function `create_unitigs` that, given a *DBG* of order k , outputs all the unitigs of the graph.

► **Question 11.** What is the sum of the size of the recovered unitigs, for each read dataset and each $t \in \{1, 2, 3, 4\}$?

(4) ■ (Bonus) “Spectrum” Preserving String Set

Recall that a SPSS of a set of kmers K is a set S of strings whose kmers are exactly those in K . By construction, the set of unitigs is a SPSS.

► **Question 12.** Write a function `is_SPSS`, that takes as input a set of kmers and a candidate SPSS, and check whether it is indeed a SPSS.

► **Question 13.** Develop, and document, heuristics to get the smallest SPSS possible for the kmers present as nodes in the previous de Bruijn graphs. How low can you go?

